1 Solve the Tic-Tac-Toe problem using the Depth First Search technique.

```
W = [(0,1,2), (3,4,5), (6,7,8), (0,3,6), (1,4,7), (2,5,8), (0,4,8), (2,4,6)]

def win(b, p): return any(all(b[i]==p for i in w) for w in W)


def dfs(b, ai):

    if win(b, 'O'): return 1

    if win(b, 'X'): return -1

    if ' ' not in b: return 0

    s = [(dfs(b[:i] + ['O' if ai else 'X'] + b[i+1:], not ai), i) for i in range(9) if b[i] == ' ']

    return max(s)[0] if ai else min(s)[0]


b = [' '] * 9

while ' ' in b and not win(b, 'O') and not win(b, 'X'):

    b[int(input("Enter position (1-9): ")) - 1] = 'X'

    if ' ' in b and not win(b, 'X'):

        best_move = max([(dfs(b[:i] + ['O'] + b[i+1:], False), i) for i in range(9) if b[i] == ' '])[1]

        b[best_move] = 'O'

    print('\n-+-+-\n'.join(f"{b[i]}|{b[i+1]}|{b[i+2]}" for i in (0, 3, 6)))


print("AI Wins" if win(b, 'O') else "Player Wins" if win(b, 'X') else "Draw")
```

2 Show that the 8-puzzle states are divided into two disjoint sets, such that any state is reachable from any other state in the same set, while no state is reachable from any state in the other set.

```
def get_inversion_parity(state):
    tiles = [tile for tile in state if tile != 0]
    inversions = sum(1 for i in range(len(tiles)) for j in range(i + 1, len(tiles)) if tiles[i] > tiles[j])
    return inversions % 2


def is_reachable(state1, state2):
    # If parities match, they are in the same disjoint set
    return get_inversion_parity(state1) == get_inversion_parity(state2)
state_A = [int(x) for x in input("Enter State A separated by spaces (e.g., 1 2 3 4 5 6 7 8 0): ").split()]
state_B = [int(x) for x in input("Enter State B separated by spaces: ").split()]


if is_reachable(state_A, state_B):
    print("\nResult: TRUE - They are in the SAME disjoint set (reachable from one another).")
else:
    print("\nResult: FALSE - They are in DIFFERENT disjoint sets (impossible to reach).")
```

3 To represent and evaluate different scenarios using predicate logic and knowledge rules.

```
# 1. Define facts & Store in Knowledge Base
knowledge_base = {
    "Alice": {"attendance": 80, "marks": 75},
    "Bob": {"attendance": 65, "marks": 80},
    "Charlie": {"attendance": 90, "marks": 45},
    "David": {"attendance": 78, "marks": 60}
}

# 2. Define predicates
def good_attendance(student): return knowledge_base[student]["attendance"] >= 75
def good_marks(student): return knowledge_base[student]["marks"] >= 50

# 3. Create rules & Match rules with facts
def is_eligible(student):
    return good_attendance(student) and good_marks(student)

# 4. Apply inference mechanism & Generate conclusions
print("=== Predicate Logic Evaluation ===")
for student in knowledge_base:
    status = "Eligible" if is_eligible(student) else "Not Eligible"
    print(f"{student}: {status}")
```

4 To apply the Find-S and Candidate Elimination algorithms to a concept learning task and compare their inductive biases and outputs.

```
# 1. Load dataset (Attendance, Internal Marks, Eligibility)
dataset = [['High', 'Good', 'Yes'], ['High', 'Poor', 'No'], ['Low', 'Good', 'No'], ['High', 'Good', 'Yes']]

# 2. Separate attributes and target labels
X, Y = [d[:-1] for d in dataset], [d[-1] for d in dataset]

# 3. Initialize most specific hypothesis (S) and general (G)
S = X[0][:]
G = [['?'] * len(S)]

# 4. Read training examples
for i in range(len(X)):
    # 5. For positive examples: generalize hypothesis S, remove inconsistent G
    if Y[i] == 'Yes':
        S = [S[j] if S[j] == X[i][j] else '?' for j in range(len(S))]
        G = [g for g in G if all(g[k] == '?' or g[k] == X[i][k] for k in range(len(S)))]

    # 6. For negative examples: update/specialize boundaries of G
    else:
        new_G = []
        for g in G:
            for j in range(len(S)):
                if g[j] == '?' and S[j] != '?' and S[j] != X[i][j]:
                    new_G.append(g[:j] + [S[j]] + g[j+1:])
        G = new_G
```

```
print(f"Find-S Hypothesis: {S} \n")


print("Candidate Elimination  Hypothesis ")
print("Final Specific Hypothesis (S):", S)
print("Final General Hypothesis (G):", G)
```

5 To construct a decision tree using the ID3 algorithm on a simple classification dataset

```
import pandas as pd

from sklearn.tree import DecisionTreeClassifier, export_text

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score


data = {

    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain', 'Overcast', 'Sunny', 'Sunny', 'Rain'],

    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool', 'Cool', 'Mild', 'Mild', 'Mild'],

    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal', 'Normal', 'High', 'Normal', 'Normal'],

    'Windy': ['False', 'True', 'False', 'False', 'False', 'True', 'True', 'False', 'False', 'True'],

    'PlayTennis': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No', 'Yes', 'Yes']

}


df = pd.DataFrame(data)


X = pd.get_dummies(df.drop('PlayTennis', axis=1))  # Convert categorical to numeric

y = df['PlayTennis']


X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)


clf = DecisionTreeClassifier(criterion='entropy', random_state=42)

clf.fit(X_train, y_train)


y_pred = clf.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}\n")

tree_rules = export_text(clf, feature_names=list(X.columns))
print(tree_rules)
```

6 To assess how the ID3 algorithm performs on datasets with varying characteristics and complexity, examining overfitting, underfitting, and decision tree depth.

```
# ID3 Decision Tree: compare depth & overfitting on datasets
from sklearn.datasets import load_iris, load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

datasets = [("Iris", load_iris()), ("Wine", load_wine())]

for name, data in datasets:
    X_train, X_test, y_train, y_test = train_test_split(
        data.data, data.target, test_size=0.3, random_state=42)

    model = DecisionTreeClassifier(criterion="entropy")   # ID3
    model.fit(X_train, y_train)

    train_acc = model.score(X_train, y_train)
    test_acc = model.score(X_test, y_test)

    print(f"\nDataset: {name}")
    print("Tree Depth:", model.get_depth())
    print("Train Accuracy:", round(train_acc, 2))
    print("Test Accuracy :", round(test_acc, 2))

    if train_acc > test_acc:
        print("Possible Overfitting")
    else:
        print("Good Generalization / Underfitting")
```

7 To examine different types of machine learning approaches (Supervised, Unsupervised, Semi-supervised, and Reinforcement Learning) by setting up a basic classification problem and exploring how each type applies differently

```
X = [[1], [2], [3], [8], [9], [10]]
y = [0, 0, 0, 1, 1, 1]
# 1. SUPERVISED LEARNING

print("\nSUPERVISED LEARNING")

# Simple prediction using labeled data
def supervised_predict(value):

    if value < 5:
        return 0
    else:
        return 1

print("Prediction for 2 :", supervised_predict(2))
print("Prediction for 9 :", supervised_predict(9))

# 2. UNSUPERVISED LEARNING

print("\nUNSUPERVISED LEARNING")

# Simple clustering without labels
for value in X:

    if value[0] < 5:
        cluster = "Cluster A"
    else:
```

```
        cluster = "Cluster B"

    print(value[0], "->", cluster)

# 3. SEMI-SUPERVISED LEARNING

print("\nSEMI-SUPERVISED LEARNING")

# Some data labeled, some unlabeled
labeled_data = {1: 0, 9: 1}

unlabeled = [2, 3, 8, 10]

for value in unlabeled:

    if value < 5:
        predicted_label = 0
    else:
        predicted_label = 1

    print("Value :", value,
          "Predicted Label :", predicted_label)

 # 4. REINFORCEMENT LEARNING

print("\nREINFORCEMENT LEARNING")

score = 0

actions = ["Correct", "Wrong", "Correct"]
```

```
for action in actions:

    if action == "Correct":
        score += 10
        print("Reward +10")
    else:
        score -= 5
        print("Penalty -5")

print("Final Score :", score)
```

8 To understand how Find-S and Candidate Elimination algorithms search through the hypothesis space in concept learning tasks, and to observe the role of inductive bias in shaping the learned concept.

```
# Training data
data = [
    (['Sunny','Warm','Normal'], 'Yes'),
    (['Sunny','Warm','High'], 'Yes'),
    (['Rainy','Cold','High'], 'No'),
    (['Sunny','Warm','Normal'], 'Yes')
]

# Find-S
S = ['0'] * len(data[0][0])

for x, y in data:
    if y == 'Yes':
        for i in range(len(S)):
            if S[i] == '0':
                S[i] = x[i]
            elif S[i] != x[i]:
                S[i] = '?'

print("Find-S Hypothesis:", S)

# Candidate Elimination
G = [['?'] * len(S)]

for x, y in data:
    if y == 'Yes':
        G = [g for g in G if all(g[i] == '?' or g[i] == x[i]
```

```
                                for i in range(len(x)))]
    else:
        newG = []
        for g in G:
            for i in range(len(x)):
                if g[i] == '?' and S[i] != x[i]:
                    h = g.copy()
                    h[i] = S[i]
                    newG.append(h)
        G = newG

print("Specific Boundary S:", S)
print("General Boundary G:", G)
```

9 To go through all stages of a real-life machine learning project, from data collection to model fine-tuning, using a regression dataset like the "California Housing Prices."

```
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import RidgeCV
from sklearn.metrics import mean_squared_error, r2_score

# 1-4. Load Dataset & Handle Missing (Sklearn dataset is pre-cleaned)
housing = fetch_california_housing()
X, y = housing.data, housing.target

# 5-6. Feature Engineering (Scaling) & Train/Test Split
X_scaled = StandardScaler().fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
test_size=0.2, random_state=42)

# 7-9. Train Model, Fine-Tune Parameters (done automatically by RidgeCV),
and Predict
model = RidgeCV(alphas=[0.1, 1.0, 10.0]).fit(X_train, y_train)
y_pred = model.predict(X_test)

# 10. Evaluate Performance
mse = mean_squared_error(y_test, y_pred)
print(f"MSE: {mse:.4f} | RMSE: {np.sqrt(mse):.4f} | R2 Score:
{r2_score(y_test, y_pred):.4f}\n")
# Display Feature Weights
print("Feature Weights:")
for name, weight in zip(housing.feature_names, model.coef_):
    print(f"{name}: {weight:.4f}")
```

10 To perform binary and multiclass classification on the MNIST dataset, analyze performance metrics, and perform error analysis.

```
import numpy as np
# Each row represents a simple image pattern
# Values:
# 0 = White pixel
# 1 = Black pixel
# 4x4 simplified digit patterns
X = np.array([
# Digit 0
[1,1,1,1,
1,0,0,1,
1,0,0,1,
1,1,1,1],

    # Digit 1
    [0,1,0,0,
     1,1,0,0,
     0,1,0,0,
     1,1,1,0],

    # Digit 2
    [1,1,1,0,
     0,0,1,0,
     1,1,1,0,
     1,0,0,0],

    # Digit 3
    [1,1,1,0,
     0,0,1,0,
```

```
     1,1,1,0,
     0,0,1,0],

    # Digit 4
    [1,0,1,0,
     1,0,1,0,
     1,1,1,0,
     0,0,1,0]

])
# Labels for digits
y = np.array([0,1,2,3,4])
# =========================================================
# DISPLAY DATASET
# =========================================================
print("========== SIMPLE MNIST-LIKE DATASET ==========\n")
for i in range(len(X)):
    print("Digit Label :", y[i])
    # Convert 1D vector into 4x4 image
    image = X[i].reshape(4,4)
    print(image)
    print()
# =========================================================
# DATASET INFORMATION
# =========================================================
print("Dataset Shape :", X.shape)
print("Number of Labels :", len(y))
```